

Love Plan

SUMMARY

Produis une véritable stratégie de sécurité applicable à une application Next.js + Supabase. Priorité absolue : La confiance des utilisateurs est plus importante que la rapidité de développement.

TECHNICAL SKILLS

Applications SaaS
Next.js
Supabase
PostgreSQL
Row Level Security
Authentification
Protection des données personnelles
Supabase Auth
Email + mot de passe
OAuth extensible
Réinitialisation de mot de passe
Confirmation email
Gestion de sessions
Refresh token
Limitation des tentatives
Protection contre brute force
Validation serveur
Row Level Security (RLS)
PostgreSQL function: `is_couple_member(couple_id)`
Supabase Storage
Signed URLs temporaires
Token aléatoire cryptographiquement sécurisé
Rate limiting
Quotas
Validation inputs
API Security
Zod
Server-side validation
Middleware
Audit log
HTTPS

OBJECTIF PRINCIPAL

Garantir qu'un utilisateur ne puisse jamais accéder aux données privées d'un autre utilisateur ou d'un autre couple, même en modifiant une URL, un ID, une requête API ou un payload frontend. Le backend doit toujours protéger les données. Ne jamais faire confiance au frontend pour gérer les permissions.

MODÈLE DE SÉCURITÉ DES COUPLES

Structure recommandée : users → profiles → couple_members → couples. Chaque utilisateur possède user_id. Un utilisateur peut appartenir à un couple actif. Les données communes doivent être liées à couple_id (projects, journal_entries, events). Règle principale : un utilisateur peut accéder à une ressource seulement s'il est membre du couple associé.

ROW LEVEL SECURITY

Activer RLS sur toutes les tables publiques contenant des données utilisateur. Créer une fonction PostgreSQL sécurisée is_couple_member(couple_id) qui vérifie si auth.uid() est présent dans couple_members pour le couple concerné. Concept : SELECT EXISTS (SELECT 1 FROM couple_members WHERE couple_id = target_couple_id AND user_id = auth.uid()). Toutes les politiques doivent s'appuyer sur ce principe.

MATRICE DES PERMISSIONS

PROFILE : utilisateur lit/modifie son profil ; autre utilisateur lecture limitée uniquement si nécessaire dans son couple. COUPLE : membre lecture/modification selon rôle ; non-membre aucun accès. PROJECTS : membre lecture/création/modification ; non-membre aucun accès. CONTRIBUTIONS : membre lecture selon règles du projet, création uniquement pour lui-même, modification limitée. JOURNAL : PRIVATE visible uniquement au créateur ; SHARED visible aux membres du couple. AI CONVERSATIONS : PRIVATE uniquement propriétaire ; SHARED membres autorisés du couple.

SUPABASE STORAGE

Ne jamais stocker les fichiers dans des buckets publics. Créer des buckets privés : avatars, journal-media, project-media, surprise-media. Utiliser des chemins structurés (exemple : couples/{couple_id}/journal/{file_id}). Les politiques Storage doivent vérifier le propriétaire ou l'appartenance au couple. Utiliser des signed URLs temporaires.

INVITATIONS PARTENAIRE

Créer un système sensible avec token aléatoire cryptographiquement sécurisé, expiration, usage unique, possibilité d'annulation. Table partner_invitations : id, inviter_id, couple_id, token_hash, expires_at, accepted_at, status. Ne jamais stocker le token brut.

SÉPARATION DU COUPLE

Prévoir un système sécurisé : 1) confirmation explicite, 2) réauthentification si nécessaire, 3) déconnexion des comptes, 4) révocation des accès, 5) traitement des données communes, 6) conservation ou suppression selon politique définie, 7) audit log. Les données privées doivent rester privées. Les données communes suivent une politique claire. Ne jamais supprimer automatiquement des données sensibles sans confirmation.

SÉCURITÉ LOVE AI

Avant d'envoyer un contexte à l'IA, vérifier : utilisateur authentifié, autorisation, nécessité des données, suppression des informations sensibles inutiles. Ne jamais envoyer tokens, secrets, passwords, informations techniques internes. Mettre en place rate limiting, quotas, validation inputs, limitation taille requêtes.

API SECURITY

Toutes les API doivent vérifier authentication/authorization, valider inputs, limiter les requêtes, ne jamais exposer de clés secrètes, utiliser des erreurs génériques côté client, logger les événements techniques importants. Utiliser Zod, server-side validation, middleware, rate limiting.

AUDIT LOG

Créer la table security_audit_logs. Exemples d'événements : LOGIN, LOGOUT, INVITATION_CREATED, INVITATION_ACCEPTED, COUPLE_DISCONNECTED, PROJECT_DELETED, ACCOUNT_DELETED, SENSITIVE_DATA_EXPORT. Ne jamais enregistrer inutilement le contenu des messages privés.

TESTS DE SÉCURITÉ

TEST 1 : utilisateur A tente d'accéder à un projet du Couple B → 403 / Aucun accès. TEST 2 : utilisateur A modifie manuellement un project_id → refus. TEST 3 : utilisateur non authentifié → redirection login. TEST 4 : ancien partenaire → accès révoqué. TEST 5 : URL média privée → impossible sans URL signée valide. TEST 6 : invitation expirée → refus. TEST 7 : utilisateur tente d'accéder à une conversation IA privée → refus.

CHECKLIST AVANT PRODUCTION

RLS activé ; toutes les politiques testées ; secrets hors frontend ; storage privé ; signed URLs ; validation serveur ; rate limiting ; HTTPS ; logs sécurisés ; gestion erreurs ; tests accès non autorisé ; suppression compte ; export données ; gestion séparation.

INSTRUCTION FINALE

Commencer par : 1) Threat Model, 2) Architecture sécurité, 3) Modèle de permissions, 4) Politiques RLS détaillées, 5) Sécurité Storage, 6) Sécurité API, 7) Sécurité IA, 8) Gestion séparation, 9) Plan de tests, 10) Checklist production.